SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING) Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	**Sameer Ahmed G**	**Reg. No.:**	**192421154**
Section / Batch:	**B Tech IT**	**Date:**	**28-07-2026**

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Code of Conduct

I Sameer Ahmed G (192421154) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sameer Ahmed G

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Debugging and Optimization of Brute Force String Matching Algorithm

Aim

To identify and correct logical errors in the brute force string matching algorithm, optimize the implementation for proper match detection, and analyze its time complexity.

Problem Statement

The given brute force string matching program incorrectly reports a match whenever the last compared character matches, even if previous characters in the same alignment do not. The program must be debugged so that a match is reported only when the complete pattern matches the text consecutively.

Errors Identified

1. The condition if (text[i + j - 1] == pattern[j - 1]) is logically incorrect because it checks only the last compared characters instead of verifying the entire pattern.
2. If a mismatch occurs before the last character, the program may still print "Match".
3. Accessing j-1 can become invalid in some situations, leading to incorrect behavior.
4. The success condition should depend on whether all characters of the pattern have been matched.

Debugging Process

1. Analyze the nested loops.
2. Verify where the inner loop terminates.
3. Observe that if all characters match, the loop exits with j == m.
4. Replace the incorrect condition with if (j == m).
5. Test the program using matching and non-matching patterns to ensure correctness.

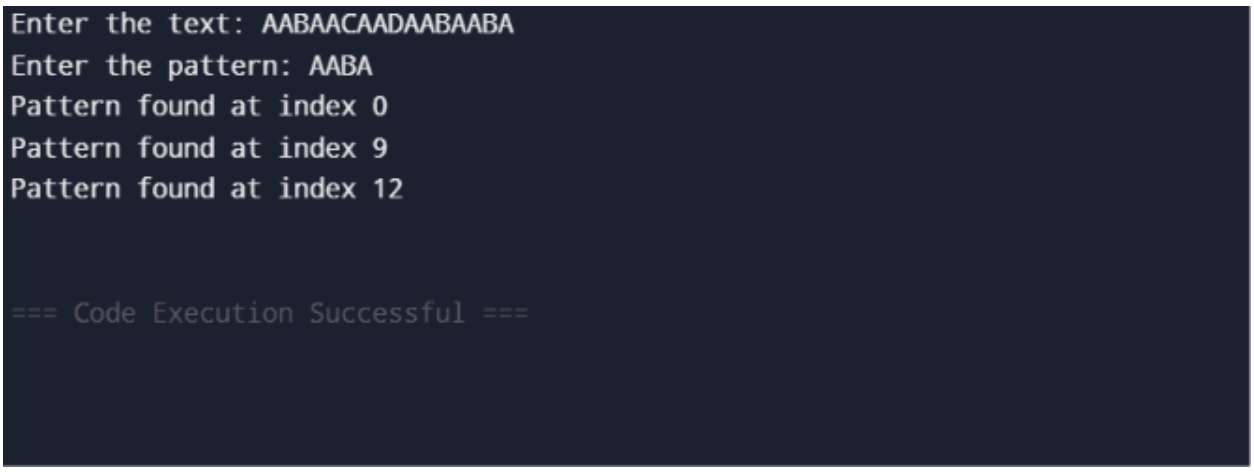
Optimized C Program

```
#include <stdio.h>
#include <string.h>

// Function to perform Brute Force String Matching
void stringMatch(char text[], char pattern[])
{
   int n = strlen(text);
   int m = strlen(pattern);
   int i, j;
   int found = 0;

   // Check every possible alignment
   for(i = 0; i <= n - m; i++)
   {
      // Compare pattern with text
      for(j = 0; j < m; j++)
      {
         if(text[i + j] != pattern[j])
            break;
      }

      // Match found only if all characters matched
      if(j == m)
      {
         printf("Pattern found at index %d\n", i);
         found = 1;
      }
   }

   if(found == 0)
      printf("Pattern not found\n");
}

int main()
{
   char text[100], pattern[100];

   printf("Enter the text: ");
```

```
    scanf("%s", text);

    printf("Enter the pattern: ");
    scanf("%s", pattern);

    stringMatch(text, pattern);

    return 0;
}
```

Sample Input 1
Enter the text: AABAACAADAABAABA
Enter the pattern: AABA

Sample Output 1
Pattern found at index 0
Pattern found at index 9
Pattern found at index 12



Sample Input 2
Enter the text: HELLOWORLD
Enter the pattern: JAVA

Sample Output 2
Pattern not found

```
Enter the text: HELLOWORLD
Enter the pattern: Java
Pattern not found


=== Code Execution Successful ===
```

Justification of Corrections

The original program incorrectly checked only the last compared characters after the inner loop. This caused false matches whenever the final compared characters were equal despite earlier mismatches. The corrected version verifies whether the entire pattern has been matched by checking j == m, ensuring that every character matches consecutively before reporting success.

Optimization

The optimized implementation introduces a found flag to report when no match exists and replaces the incorrect comparison with a direct loop completion check. This improves code clarity, correctness, and reliability while preserving the brute force searching technique.

Time Complexity

Case	Time Complexity
Best Case	O(n)
Average Case	O(n × m)
Worst Case	O(n × m)
Space Complexity	O(1)

Analysis

The corrected algorithm compares each possible alignment of the pattern with the text character by character. It stops comparison immediately when a mismatch occurs, avoiding unnecessary comparisons. Although the brute force approach remains unchanged, the corrected success condition guarantees accurate match reporting and eliminates false positives.

Conclusion

The debugging process successfully removed the logical error responsible for incorrect match reporting. The optimized brute force string matching algorithm now reports a match only when the complete pattern matches the text consecutively. The algorithm maintains its original brute force behavior with a worst-case time complexity of **O(n × m)** while producing correct and reliable results.